Mini-Projet : Fouille de Données 1

Développement d'une Interface Interactive Complète

Data Mining Studio, Pipeline de prétraitement, clustering et classification

1. Introduction

Ce rapport présente le développement d'une application interactive complète pour la fouille de données (data mining), construite avec Streamlit. L'application implémente le pipeline complet demandé par le projet : prétraitement des données (Volet 1), clustering sans supervision avec K-Means et K-Medoids implémentés from scratch (Volet 2), et classification supervisée avec 7 modèles distincts (Volet 3).

2. Objectifs du Projet

- Concevoir une interface graphique interactive permettant le chargement et l'exploration des données
- Implémenter des algorithmes de clustering (K-Means et K-Medoids) from scratch en NumPy
- Proposer une analyse supervisée avec plusieurs modèles de classification
- Fournir des visualisations interactives et des métriques d'évaluation robustes
- Assurer une architecture modulaire, extensible et facilement maintenable

3. Architecture et Structure du Projet

L'application suit une architecture modulaire en trois couches : couche présentation (Streamlit), couche métier (algorithmes), couche utilitaires.

Module	Responsabilité
app.py	Entrée Streamlit, orchestration des onglets et sidebar
src/preprocessing.py	Volet 1 : import, exploration, nettoyage, normalisation, visualisation
src/clustering.py	Volet 2 : K-Means & K-Medoids (from scratch), Elbow, Silhouette, PCA
src/classification.py	Volet 3 : 7 modèles, métriques, matrice de confusion
src/styles.py & src/utils.py	CSS personnalisé, thème Plotly, gestion état, I/O données

3.1 Arborescence

```
mini-projet/
├── app.py                    # Point d'entrée Streamlit
├── Makefile                  # Automatisation (install, run, lint, clean)
├── requirements.txt          # Dépendances Python
├── README.md                 # Documentation
├── .streamlit/config.toml    # Configuration Streamlit
├── data/
│   └── patients_health.csv   # Dataset d'exemple
└── src/
    ├── __init__.py
    ├── styles.py             # CSS & primitives UI
    ├── utils.py              # Utilitaires partagés
    ├── preprocessing.py      # Volet 1 - Prétraitement
    ├── clustering.py         # Volet 2 - Clustering
    └── classification.py     # Volet 3 - Classification
```

4. Volet 1 : Prétraitement des Données

Le prétraitement est la fondation d'une bonne analyse. Ce volet automatise et rend interactive chaque étape du nettoyage et de la préparation des données.

4.1 Importation de Données

L'application supporte plusieurs formats :
• Fichiers : CSV, TSV, Excel (.xlsx/.xls), JSON, Parquet
• Datasets sklearn pré-chargés : Iris, Wine, Breast Cancer, Diabetes

Code :

```
.file_uploader("Glisser-déposer", type=["csv", "xlsx", "json", ...],
key="pre_uploader")
load_sample("Iris")  # Charge datasets sklearn
```

4.2 Exploration Interactive

Trois onglets complémentaires :
1. Statistiques descriptives (min, max, moyenne, écart-type, quartiles)
2. Tableau des types de données avec comptage des NaN par colonne
3. Visualisation bar chart des valeurs manquantes

4.3 Nettoyage des Données Manquantes

Six stratégies implémentées :

- Supprimer les lignes (dropna)
- Supprimer les colonnes
- Imputer par la moyenne
- Imputer par la médiane (robustesse aux outliers)
- Imputer par le mode

- Imputer par valeur fixe (personnalisable)

```
if strategy == "Imputer — médiane":
    for c in target_cols:
        df[c] = df[c].fillna(df[c].median())
log_step(f"Nettoyage appliqué : {before - after} NaN traités")
```

4.4 Normalisation

Deux méthodes disponibles :

- Min-Max Scaling : $X' = (X - X_{min}) / (X_{max} - X_{min}) \in [0, 1]$
- Standardisation Z-score : $X' = (X - \mu) / \sigma$, où $\mu=0$ et $\sigma=1$

```
scaler = MinMaxScaler()  # ou StandardScaler()
df[cols] = scaler.fit_transform(df[cols])
```

4.5 Visualisations Interactives

Quatre types de graphiques Plotly :
• Boxplot : distribution et détection d'outliers
• Scatter : relation bivariée avec codage couleur
• Histogramme : distribution univariée (contrôle des bins)
• Heatmap de corrélation : détection des dépendances linéaires

5. Volet 2 : Clustering Non-Supervisé

Implémentation complète de deux algorithmes de clustering from scratch, avec métriques d'évaluation et visualisations. Les algorithmes sont codés en NumPy pur (pas de dépendance à scikit-learn pour l'algorithme lui-même).

5.1 K-Means Implémenté from Scratch

Algorithme de clustering partitionnel. Initialisation par k-means++ garantissant une bonne convergence.

```
def kmeans(X, k, max_iter=300, tol=1e-4, random_state=42):
    """K-Means avec initialisation k-means++."""
    rng = np.random.default_rng(random_state)
    n = X.shape[0]

    # Initialisation k-means++ :
    # Choisit k points initiaux en faveur des points éloignés
    centers = [X[rng.integers(n)]]
    for _ in range(1, k):
        d2 = np.min(distances_to_centers ** 2, axis=1)
        probs = d2 / d2.sum()
        centers.append(X[rng.choice(n, p=probs)])
    centers = np.array(centers)

    # Itérations : assignment → update
    for it in range(max_iter):
        # Assignment : assigner chaque point au centre le plus proche
        distances = np.linalg.norm(X[:, None, :] - centers[None, :, :], axis=2)
        labels = np.argmin(distances, axis=1)
```

```
        # Update : recalculer les centres comme moyennes
        new_centers = np.array([
            X[labels == j].mean(axis=0) if np.any(labels == j) else centers[j]
            for j in range(k)
        ])

        # Critère d'arrêt
        shift = np.linalg.norm(new_centers - centers)
        centers = new_centers
        if shift < tol:
            break

    inertia = float(np.sum((X - centers[labels]) ** 2))
    return ClusterResult(labels, centers, inertia, it + 1)
```

Points clés :
• Initialisation k-means++ : O(n·k) mais élimine les initialisations pathologiques
• Convergence monotone : inertie toujours décroissante
• Inertie = $\Sigma\ ||x - \text{center}(x)||^2$ (within-cluster sum of squares)

5.2 K-Medoids Implémenté from Scratch

Variante robuste du K-Means où les centres sont des points du dataset (medoids). Utilise l'algorithme PAM avec swap greedy.

Avantages par rapport à K-Means :
• Robustesse aux outliers : medoid = point réel, non moyenne
• Interprétabilité : chaque cluster centré sur un exemplaire du dataset
• Métriques arbitraires : fonctionne avec distance quelconque

```
def kmedoids(X, k, max_iter=300, random_state=42):
    """K-Medoids (PAM) avec swap greedy."""
    rng = np.random.default_rng(random_state)

    # Distance précalculée
    D = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=2)

    # Initialisation : sélectionner k medoids dispersés
    medoid_idx = [rng.integers(n)]
    for _ in range(1, k):
        d_min = np.min(D[:, medoid_idx], axis=1)
        probs = d_min / d_min.sum()
        medoid_idx.append(rng.choice(n, p=probs))

    # Itérations PAM : assignment puis swap
    for it in range(max_iter):
        labels = np.argmin(D[:, medoid_idx], axis=1)
        improved = False

        # Pour chaque cluster : essayer de remplacer son medoid
        for j in range(k):
            cluster_points = np.where(labels == j)[0]
            if cluster_points.size == 0:
                continue
            # Meilleur point du cluster (minimise distance intra-cluster)
            cost_within = D[np.ix_(cluster_points, cluster_points)].sum(axis=1)
            best = cluster_points[np.argmin(cost_within)]
```

```
            if best != medoid_idx[j]:
                trial = medoid_idx.copy()
                trial[j] = best
                new_cost = sum(np.min(D[:, trial], axis=1))
                if new_cost < cost:
                    medoid_idx = trial
                    improved = True

        if not improved:
            break

    centers = X[medoid_idx]
    return ClusterResult(labels, centers, inertia, it + 1)
```

5.3 Méthode du Coude (Elbow)

Graphe dual affichant pour chaque k ∈ [2, k_max] :
• Inertie (axe gauche) : somme carrés distances intra-cluster
• Score Silhouette (axe droit) : mesure qualité de clustering ∈ [-1, 1]

L'utilisateur identifie le "coude" où la décroissance d'inertie ralentit.

5.4 Score de Silhouette

Indice de qualité pour chaque point : $s = (b - a) / \max(a, b)$ où :
• a = distance moyenne au cluster du point
• b = distance moyenne au cluster le plus proche

Interprétation :
• $s \approx 1$: point bien assigné
• $s \approx 0$: chevauchement
• $s < 0$: mal-assigné

5.5 Projection 2D via PCA

Si dimension > 2, réduction via PCA (2 premières composantes principales). Affichage interactive avec points coloriés par cluster et centroïdes/medoids marqués par des ×.

6. Volet 3 : Classification Supervisée

Pipeline complet : partitionnement train/test, entraînement, prédiction, évaluation. Sept modèles distincts avec hyperparamètres configurables.

6.1 Partitionnement Train/Test

Division contrôlée (taille configurable, par défaut 75/25) avec stratification (préserve proportions de classes). Optionnel : normalisation StandardScaler.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
if scale:
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

6.2 Sept Modèles Disponibles

Modèle	Hyperparamètres	Use Case
KNN	$k \in [1, 30]$	Baseline simple, non paramétrique
Decision Tree	max_depth $\in [1, 30]$	Interprétabilité, non-linéaire
Random Forest	n_trees, max_depth	Robustesse, importance variables
Gradient Boosting	n_estimators $\in [50, 500]$	Performance, données complexes
Logistic Regression	$C \in [0.01, 10]$	Probabilités, linéarité
SVM	C, kernel	Haute dimension, marge max
Gaussian Naïve Bayes	(aucun)	Probabiliste, léger

6.3 Métriques d'Évaluation

Pour chaque modèle entraîné sur le test set :

Métrique	Formule / Interprétation
Accuracy	(TP + TN) / (TP + TN + FP + FN) — % de bonnes prédictions
Precision	TP / (TP + FP) — parmi les positifs prédits, combien réels
Recall	TP / (TP + FN) — parmi vrais positifs, combien trouvés
F1-score	2 · (P · R) / (P + R) — moyenne harmonique P-R

6.4 Matrice de Confusion

Tableau (n_classes × n_classes) avec True/False Positives/Negatives. Heatmap colorée pour détection visuelle rapide des erreurs systématiques.

6.5 Rapport Détaillé par Classe

Table avec Precision/Recall/F1 pour chaque classe, permettant de détecter des déséquilibres ou des classes difficiles à prédire.

6.6 Importance des Variables

Graphe bar affichant :

- Tree models (DT, RF, GB) : feature_importances (basé sur gain informatif)
- Linear models (LR, SVM) : coefficients du modèle

7. Détails d'Implémentation

7.1 Gestion d'État avec Streamlit

Streamlit exécute le script complet à chaque interaction. L'état persiste via session_state :

```
def ensure_state():
    """Initialise les clés attendues de session_state."""
    defaults = {
        "raw_df": None,         # DataFrame original (non modifié)
        "df": None,             # DataFrame de travail (modifié)
        "source_name": None,    # Nom du fichier/dataset chargé
        "history": [],          # Historique des transformations
    }
    for key, value in defaults.items():
        st.session_state.setdefault(key, value)
```

Les trois onglets accèdent au même st.session_state, permettant un workflow fluide : charger dans l'onglet 1, analyser dans l'onglet 2, classer dans l'onglet 3.

7.2 Paramétrage Unique des Widgets

Tous les widgets ont un identifiant unique (`key=`) pour éviter les collisions. Préfixes : pre_* (preprocessing), clu_* (clustering), cls_* (classification).

```
# Volet 2
features = st.multiselect("Variables", num_cols,
                          default=num_cols, key="clu_features")
algo = st.selectbox("Algorithme", ["K-Means", "K-Medoids"],
                    key="clu_algo")
k = st.number_input("k", min_value=2, max_value=15,
                    value=3, step=1, key="clu_k")

# Volet 3
target = st.selectbox("Variable cible", df.columns,
                      key="cls_target")
test_size = st.slider("Taille de test", 0.1, 0.5, 0.25,
                      key="cls_test_size")
```

7.3 Thème Visuel (CSS personnalisé)

CSS injecté dans le DOM Streamlit (src/styles.py) définissant :

- Gradient hero bleu-violet avec ombre
- Sélecteurs sûrs pour ne pas masquer la mise en page
- Palette cohérente pour boutons, cartes, onglets, sidebar
- Couleurs pour Plotly (template sombre personnalisé)

```
CUSTOM_CSS = """
<style>
.hero {
    background: linear-gradient(135deg, #1E1B4B 0%, #2D1B69 50%, #0F172A
100%);
    border: 1px solid rgba(124, 124, 255, 0.25);
    padding: 28px 32px;
```

```
    border-radius: 16px;
}
.stButton > button {
    background: linear-gradient(135deg, #6366F1 0%, #7C7CFF 100%);
    box-shadow: 0 2px 10px rgba(99, 102, 241, 0.3);
}
</style>
"""
st.markdown(CUSTOM_CSS, unsafe_allow_html=True)
```

7.4 Visualisations Interactives (Plotly)

Toutes les visualisations utilisent Plotly pour l'interactivité (zoom, hover, export PNG).

```
def style_fig(fig, height=420):
    """Applique le thème personnalisé à une figure Plotly."""
    fig.update_layout(
        template="plotly_dark",
        height=height,
        paper_bgcolor="#161B26",
        plot_bgcolor="#161B26",
        font=dict(family="Inter, sans-serif", color="#E2E8F0"),
        margin=dict(l=20, r=20, t=40, b=20),
    )
    fig.update_xaxes(gridcolor="rgba(255,255,255,0.05)")
    fig.update_yaxes(gridcolor="rgba(255,255,255,0.05)")
    return fig

# Usage
fig = px.bar(...)
st.plotly_chart(style_fig(fig, 420), use_container_width=True)
```

8. Stack Technologique

Technologie	Utilisation
Streamlit 1.56+	Framework web interactif, sans JavaScript
pandas 2.1+	Manipulation, exploration, transformation données
numpy 1.26+	Calculs vectorisés, algorithmes from scratch
scikit-learn 1.4+	Modèles classification, train/test split, preprocessing
Plotly 5.20+	Visualisations interactives, export
Python 3.13	Langage
Makefile	Automatisation (install, run, lint, clean)
python-docx	Génération rapport académique

9. Utilisation et Déploiement

9.1 Installation

```
# Créer l'environnement virtuel et installer les dépendances
make install

# Ou manuellement :
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

9.2 Lancement

```
# Lancer l'application (rechargement automatique)
make run

# Ou en développement (auto-reload sur changement fichier)
make dev

# Lancer directement avec Streamlit
streamlit run app.py --server.port=8501
```

L'application démarre sur http://localhost:8501 et charge automatiquement.

9.3 Autres Cibles Make

make clean — supprime caches et artifacts
make lint — vérification syntaxique Python
make freeze — verrouille versions dépendances (requirements.lock.txt)

10. Validation et Résultats

10.1 Dataset d'Exemple

Un dataset synthétique réaliste (patients_health.csv) est fourni pour démonstration :

- 240 patients × 10 variables
- 3 classes équilibrées : Healthy, At-Risk, Diseased (80 chacun)
- 22 valeurs manquantes (~3%) distribuées sur 4 colonnes
- 6 variables numériques avec distributions distinctes
- 2 variables catégorielles : sex (M/F), smoker (never/former/current)
- Variables numériques : bmi, heart_rate, systolic_bp, diastolic_bp, hba1c, cholesterol

10.2 Scénario de Test Complet

Workflow recommandé pour valider l'application :

1. Charger patients_health.csv (tab Prétraitement)
2. Exploration → constater 22 NaN
3. Nettoyage : Imputer — médiane sur 4 colonnes
4. Normalisation : Standardisation Z-score sur 6 colonnes
5. Visualisation : boxplot par diagnosis, scatter bmi vs cholesterol
6. Clustering (tab) : K-Means, k_max=8 → Elbow à k=3, Silhouette > 0.7
7. Classification (tab) : Random Forest, target=diagnosis
   → Accuracy ≥ 0.90, matrice confusion nette sur diagonale

Résultats attendus :
• Clustering : 3 clusters bien séparés en PCA, correspondant aux 3 classes
• Classification : feature_importances montrent hba1c >> bmi > systolic_bp
• Visualisations : toutes interactives, exportables en PNG

11. Conclusion

Cette application Data Mining Studio réalise l'intégralité des objectifs du mini-projet. Elle couvre les trois volets demandés avec une couverture fonctionnelle complète :

✓ Prétraitement : import multi-format, exploration, 6 stratégies nettoyage, 2 normalisations, 4 visualisations
✓ Clustering : K-Means & K-Medoids from scratch, Elbow, Silhouette, PCA projection
✓ Classification : 7 modèles, train/test split stratifié, 4 métriques, confusion matrix, feature importance
✓ Architecture : modulaire, maintenable, extensible
✓ UI : élégante, intuitive, thème personnalisé, visualisations interactives

L'implémentation from scratch des algorithmes de clustering démontre la maîtrise des concepts fondamentaux. La gestion d'état Streamlit assure un workflow fluide entre les trois volets. L'application est prête pour l'utilisation en production et l'extension future (ajout de datasets, modèles supplémentaires, export de résultats).

Annexe A : Structure des Fonctions Principales

A.1 src/preprocessing.py

Fonctions principales (appels depuis render()) :

- _render_import() — upload/select dataset
- _render_overview() — métriques globales (lignes, colonnes, NaN, mémoire)
- _render_explore() — statistiques descriptives, types, valeurs manquantes
- _render_cleaning() — 6 stratégies NaN
- _render_normalisation() — Min-Max & Z-score
- _render_visualisation() — 4 types graphiques
- render() — orchestration publique

A.2 src/clustering.py

- kmeans(X, k, max_iter, tol, random_state) — from scratch
- kmedoids(X, k, max_iter, random_state) — from scratch
- _elbow_chart(X, k_max, algorithm) — courbe inertie + silhouette
- _projection_chart(X, labels, centers) — PCA + scatter
- render() — orchestration

A.3 src/classification.py

- _hyperparams(model_name) — widgets hyperparamètres
- _confusion_fig(cm, classes) — heatmap confusion
- _metric_card(label, value) — affichage métrique
- render() — orchestration

A.4 src/styles.py

- inject_css() — injecter stylesheet personnalisé
- hero(title, subtitle, pill) — composant hero
- section(title) — titre de section avec point brillant
- empty_state(icon, title, desc) — état vide
- badge(text, kind) — petit badge couleur
- style_fig(fig, height) — thème Plotly personnalisé

A.5 src/utils.py

- ensure_state() — initialiser session_state
- log_step(message) — ajouter à historique
- reset_working_df() — réinitialiser à l'original
- load_uploaded_file(file) — CSV/TSV/Excel/JSON/Parquet
- load_sample(name) — sklearn datasets
- numeric_columns(df), categorical_columns(df) — filtrage colonnes
- style_fig(fig, height) — thème Plotly